

# La cumpărături

---

- Data publicării: 09.03.2026
- Data ultimei modificări: 09.03.2026 ([changelog](#))
- Tema (o arhivă .zip cu toate fișierele .rkt folosite în etapa curentă) se va încărca pe vmchecker [<https://vmchecker.cs.pub.ro/ui/#PP>]

## Descriere generală și organizare

---

Tema constă într-o aplicație care simulează fluxul clienților pe la casele unui magazin, și constă în **4 etape**:

- una pe care o veți rezolva după laboratorul 3 (cu deadline în ziua laboratorului 4, la ora 08:00)
- una pe care o veți rezolva după laboratorul 4 (cu deadline în ziua laboratorului 5, la ora 08:00)
- una pe care o veți rezolva după laboratorul 5 (cu deadline în ziua laboratorului 6, la ora 08:00)
- una pe care o veți rezolva după laboratorul 6 (cu deadline în ziua laboratorului 7, la ora 08:00)

Așa cum se poate observa, **ziua deadline-ului variază în funcție de semigrupa în care sunteți repartizați. Restanțierii care refac tema și nu refac laboratorul beneficiază de ultimul deadline** (deci vor avea deadline-uri în zilele de 20.03, 27.03, 03.04, 10.04).

Rezolvările tuturor etapelor pot fi trimise până la ora 08:00 în ziua laboratorului 7, dar orice exercițiu trimis după deadline-ul soft și înainte de deadline-ul hard (care este dimineața zilei laboratorului 7) se punctează cu **jumătate** din punctaj. Orice exercițiu trimis după deadline-ul hard nu se mai punctează deloc. Nota finală pe etapă se calculează conform formulei:  $n = (n1 + n2) / 2$  ( $n1$  = nota obținută înainte de deadline-ul soft;  $n2$  = nota obținută între deadline-ul soft și cel hard). Când toate submiisiile sunt înainte de deadline-ul soft, nota pe ultima submisie este și nota finală (întrucât  $n1 = n2$ ).

În fiecare etapă, veți folosi ce ați învățat în săptămâna anterioară pentru a perfecționa aplicația.

Pentru fiecare etapă veți primi un **schelet de cod** (dar rezolvarea se bazează în mare măsură pe rezolvările anterioare). Enunțul din această pagină este menit să descrie pe scurt aplicația și să ofere exemple de rulare a funcțiilor mai complexe. Dacă preferați, puteți rezolva tema utilizând doar indicațiile din schelet.

## Etapă 1

---

În această etapă presupunem că magazinul are fix 4 case ("counters" în engleză): C1, C2, C3, C4. Fiecare casă este reprezentată ca o structură:

```
(define-struct counter (index tt queue))
```

- **index**
  - în cazul nostru, este un număr de la 1 la 4 (1 pentru C1, 2 pentru C2, etc.)
- **tt**
  - vine de la "total time", și reprezintă timpul total de așteptare la această casă: dacă un client se așază acum la coadă, el va avea de așteptat **tt** unități de timp până să ajungă în față (pentru conveniență vom considera unitatea de timp ca fiind 1 minut, chiar dacă este nerealist)
  - depinde de numărul de produse cumpărate de clienții din coadă (1 produs = 1 minut) și de eventualele întârzieri suferite de casa respectivă
- **queue**

- este o listă de perechi (nume . nr-produse), reprezentând persoanele așezate la coadă la această casă (fiecare persoană apare în pereche cu numărul de produse cumpărate)
- ca orice coadă, funcționează pe principiul FIFO (primul element din listă corespunde primului client care s-a așezat la coadă)

Deși nu am studiat structuri la curs sau laborator, utilizarea lor este simplă (și ne ajută să avem un cod mai lizibil). Aveți aici un tutorial foarte scurt cu tot ce vă trebuie pe partea de structuri și pattern matching.

Statutul caselor diferă astfel:

- C2-C4 sunt deschise tuturor clienților
- C1 acceptă doar clienți care au cumpărat maxim ITEMS produse (ITEMS este o constantă definită în schelet)

În această etapă, simulatorul trebuie să modeleze două situații:

- când un client dorește să se așeze la coadă cu coșul său de cumpărături
- când activitatea unei case este întârziată (din diverse motive neprevăzute) cu un număr de minute

Funcțiile principale pe care va trebui să le implementați sunt:

```
(min-tt counters)
```

- min-tt determină casa din counters care are tt minim, și întoarce perechea dintre indexul acestei case și valoarea tt-ului ei)
- când are de ales între mai multe case, o va alege pe cea cu index minim
- ex:

```
(min-tt (list (counter 1 10 '()) (counter 2 12 '((ana . 12)))))
```

- tt-ul minim este 10, la casa 1
- ⇒ '(1 . 10)

```
(add-to-counter C name n-items)
```

- add-to-counter adaugă în coada casei C persoana name cu n-items produse (ceea ce se adaugă este o pereche care conține ambele informații)
- ex:

```
(add-to-counter (counter 1 10 '((dan . 10))) 'ana 12)
```

- adaugă perechea '(ana . 12) la sfârșitul cozii
- ⇒ (counter 1 22 '((dan . 10) (ana . 12)))

```
(serve requests C1 C2 C3 C4)
```

- serve primește o listă de cereri (așezări la coadă, respectiv întârzieri) și le tratează în ordine, în sensul că actualizează C1, C2, C3 și C4 pe măsură ce situația caselor evoluează

Exemplu:

```
(serve '((ana 12) (delay 1 5) (mia 2)) C1 C2 C3 C4)
```

unde presupunem că C1-C4 sunt în prezent lipsite de clienți, iar ITEMS = 5:

- ana se așază la cea mai avantajoasă casă posibilă
  - întrucât ana are 12 produse, ea se poate așeza doar la una dintre C2, C3, C4
  - toate cele 3 case sunt lipsite de clienți și niciuna nu a suferit întârzieri, deci alegem C2 pentru că are index minim
- casa 1 suferă o întârziere de 5 minute
  - o casă fără clienți poate suferi întârzieri - în sensul că un client care se așază acum la C1 trebuie să aștepte 5 minute până când cineva îl va lua în primire
- mia se așază la cea mai avantajoasă casă posibilă
  - întrucât mia are 2 produse, ea se poate așeza la orice casă
  - situația curentă a caselor este: C1 este întârziată cu 5 minute ( $tt = 5$ ), la C2 stă ana ( $tt = 12$ ), C3 și C4 nu au nici clienți, nici întârzieri ( $tt = 0$ )
  - alegem C3 pentru că, dintre casele cu  $tt$  minim, C3 are index minim
- $\Rightarrow$

```
(list
  (counter 1 5 '())
  (counter 2 12 '((ana . 12)))
  (counter 3 2 '((mia . 2)))
  (counter 4 0 '()))
```

## Etapă 2

Etapă 2 își propune exploatarea faptului că funcțiile sunt valori de ordinul întâi. Veți defini funcții curry, veți abstractiza funcții cu implementări similare, și veți folosi funcționale - atât implementări proprii, cât și funcționalele predefinite în Racket. Vă încurajăm să valorificați oportunitățile de utilizare a funcțiilor anonime și funcționalelor, inclusiv când enunțul nu impune acest lucru.

În această etapă, numărul de case din magazin nu mai este fixat. Avem:

- o listă fast-counters de case care acceptă doar clienți care au cumpărat maxim ITEMS produse
- o listă slow-counters de case deschise tuturor clienților

Pentru ca în viitor să putem determina ordinea ieșirii clienților din magazin, introducem un nou câmp în structura counter:

```
(define-struct counter (index tt et queue))
```

- et
  - vine de la "exit time", și reprezintă timpul rămas până când primul client din coadă va părăsi această casă
  - depinde de numărul de produse cumpărate de acest client (1 produs = 1 minut) și de eventualele întârzieri suferite de casă

Simulatorul trebuie să modeleze atât situațiile de la etapa anterioară, cât și două noi situații:

- când cel mai avansat client (din punct de vedere al exit time-ului) părăsește magazinul
- când este necesară deschiderea unor noi case, pentru a micșora media timpilor totali de așteptare

Inițial, veți adapta o serie de funcții de la etapa 1 la noua reprezentare (adică la numărul variabil de case și la prezența câmpului et în structură).

Apoi, funcțiile principale pe care va trebui să le implementați sunt:

```
(update f counters index)
```

- update aplică transformarea f casei din counters care are indexul index, și întoarce lista counters actualizată

Exemplu:

```
(update (λ (C) (struct-copy counter C [tt 0]))  
  (list (counter 1 2 2 '()) (counter 2 5 5 '()))  
  2)
```

⇒

```
(list (counter 1 2 2 '()) (counter 2 0 5 '()))
```

```
(remove-first-from-counter C)
```

- remove-first-from-counter scoate prima persoană din coada casei C
- tt-ul și et-ul casei C trebuie ajustate în consecință
  - orice întârziere avea casa, ea dispare
  - dispar produsele clientului care pleacă (și minutele asociate acestora)
  - nicio altă casă nu este afectată (este ca și cum ar fi trecut timpul doar pe la casa C; acest lucru se va schimba în etapa 3)

Exemplu:

```
(remove-first-from-counter (counter 1 50 5 '((ana . 3) (leo . 35) (mia . 10))))
```

⇒

```
(counter 1 45 35 '((leo . 35) (mia . 10)))
```

```
(serve requests fast-counters slow-counters)
```

- serve primește o listă de cereri (așezări la coadă, întârzieri, ieșiri de la casă, ajustări ale numărului de case) și le tratează în ordine, în sensul că actualizează casele din fast-counters și slow-counters pe măsură ce situația lor evoluează

Exemplu (pentru ITEMS = 5):

```
(serve '((ana 8) (mia 2) (mara 14) (ion 7) (remove-first) (ensure 5) (remove-first))  
  (list (empty-counter 1) (empty-counter 2))  
  (list (empty-counter 3) (empty-counter 4)))
```

- observăm că avem două case fast (pentru simplitate le numim C1 și C2) și două case slow (le numim C3 și C4)
- primele 4 cereri distribuie cei 4 clienți astfel:
  - ana la C3 (prima casă slow cu tt=0) ⇒ C3 = (counter 3 8 8 '((ana . 8)))
  - mia la C1 (prima casă fast cu tt=0) ⇒ C1 = (counter 1 2 2 '((mia . 2)))
  - mara la C4 (casa slow cu tt minim) ⇒ C4 = (counter 4 14 14 '((mara . 14)))
  - ion la C3 (casa slow cu tt minim) ⇒ C3 = (counter 3 15 8 '((ana . 8) (ion . 7)))
- remove-first scoate cel mai avansat client:
  - cel mai avansat client este mia (et=2)
  - ea este scoasă de la C1 ⇒ C1 = (counter 1 0 0 '()) (observați tt și et)

- ensure compară media timpilor totali cu 5:
  - $tt1 + tt2 + tt3 + tt4 = 0 + 0 + 15 + 14 = 29 \Rightarrow ttmed = 29 / 4 > 5$
  - se adaugă o casă slow goală (C5)  $\Rightarrow ttmed = 29 / 5 > 5$
  - se adaugă o casă slow goală (C6)  $\Rightarrow ttmed = 29 / 6 \leq 5$  (deci putem trece la cererea următoare)
- remove-first scoate cel mai avansat client:
  - cel mai avansat client este ana (et=8)
  - ea este scoasă de la C3  $\Rightarrow C3 = (\text{counter } 3 \ 7 \ 7 \ '((ion \ . \ 7)))$  (observați tt și et)

## Etapa 3

Etapa 3 subliniază importanța abstractizării. Vă veți defini propriul TDA (tip de date abstract) cu o interfață completă (un set de constructori și operatori) prin care utilizatorul poate manipula valorile tipului, independent de implementarea din spate. Apoi, voi înșivă trebuie să folosiți TDA-ul doar prin intermediul interfeței (aspect esențial pentru o dezvoltare facilă în etapa 4).

Rezolvarea etapei începe cu implementarea TDA-ului queue în fișierul **queue.rkt**.

Acest tip reprezintă o coadă (first-in-first-out) ca pe o structură:

```
(define-struct queue (left right size-l size-r))
```

- left, right
  - sunt stive (last-in-first-out, implementate ca liste Racket)
  - o adăugare în coadă este o adăugare în stiva right
  - ex adăugare:
    - adaug 1  $\Rightarrow$  right = '(1),
    - adaug 2  $\Rightarrow$  right = '(2 1)
  - o extragere din coadă este o extragere din stiva left (când left este vidă, mutăm toate elementele din right în left, apoi extragem din left)
  - mutarea este rezultatul unor operații pop (din right) + push (în left) repetate
  - ex mutare:
    - mut în left = '() din right = '(2 1)  $\Rightarrow$
    - left = '(2), right = '(1) (primul pop din right îl extrage pe 2 și îi face push în left)  $\Rightarrow$
    - left = '(1 2), right = '() (apoi pop din right îl extrage pe 1 și îi face push în left)
  - ex extragere:
    - extrag din coada cu left = '(), right = '(2 1)  $\Rightarrow$
    - left = '(1 2), right = '() (după mutarea elementelor)  $\Rightarrow$
    - left = '(2), right = '() (după extragerea primului element)
  - observați că primul element adăugat este primul element extras (first-in-first-out)
- size-l, size-r
  - sunt numere naturale, reprezentând numărul de elemente din cele două stive

Sarcina voastră este să implementați interfața TDA-ului queue:

```
empty-queue :          -> queue (constructor nular pentru o coadă goală)
queue-empty? : queue -> Bool (operator care verifică dacă o coadă este goală)
enqueue     : Elem x queue -> queue (operatorul de adăugare în coadă)
dequeue     : queue -> queue (operatorul de extragere din coadă)
top         : queue -> Elem (operatorul de vizualizare a elementului din vârful cozii)
```

Această reprezentare asigură cost amortizat  $O(1)$  pentru operațiile de enqueue și dequeue. Vom folosi acest TDA pentru câmpul queue al structurii counter, optimizând reprezentarea cu liste Racket din etapele 1 și 2.

După ce ați finalizat implementarea TDA-ului, continuați implementarea în fișierul **etapa3.rkt**.

Mai întâi, adaptați funcțiile de la etapa 2 astfel încât ele să țină cont de noua reprezentare (în care câmpul queue din structura counter este de tip coadă (queue); este o coincidență că numele câmpului coincide cu numele tipului, nu o necesitate).

În această etapă simulatorul modelează, în plus, trecerea timpului. Anterior, simulatorul trata așezările la cozi, întârzierile și deschiderile de noi case ca și cum s-ar produce în ordine, dar la un același moment de timp. Acest lucru nu corespunde realității - între evenimente este firesc să treacă timp, timp în care clienții avansează la case și, la un moment dat, părăsesc magazinul.

Funcțiile principale pe care va trebui să le implementați sunt:

```
(pass-time-through-counter minutes)
```

- este o funcție curry (aplicată parțial pe un număr de minute, va aștepta un al doilea argument de tip counter)
- odată ce și-a primit (pe rând) argumentele, pass-time-through-counter actualizează tt-ul și et-ul casei pentru a reflecta trecerea numărului dat de minute
- câmpul queue nu se modifică, deoarece intenția este să folosim această funcție doar cu timpi mai mici sau egali cu timpul până la ieșirea primului client din coadă

Exemplu:

```
((pass-time-through-counter 5)
 (counter 1
  12
  7
  (make-queue '() '((ada . 7)) 0 1)))
```

⇒

```
(counter 1 7 2 (queue '() '((ada . 7)) 0 1))
```

```
(serve requests fast-counters slow-counters)
```

- serve actualizează casele din fast-counters și slow-counters pe măsură ce situația lor evoluează, pe baza listei requests în care elementele sunt:
  - cereri (așezări la coadă, întârzieri, ajustări ale numărului de case)
  - sau
  - timpi care trec între cereri

Exemplu pentru ITEMS = 5:

```
(serve '((ana 14) (mia 2) 5 (ion 7) (delay 1 2) 7)
 (list (empty-counter 1))
 (list (empty-counter 2) (empty-counter 3)))
```

- observăm că avem o casă fast (o numim C1) și două case slow (le numim C2 și C3)
- primele două cereri distribuie cei doi clienți astfel:
  - ana la C2 ⇒ C2 = (counter 2 14 14 (queue '() '((ana . 14)) 0 1))
  - mia la C1 ⇒ C1 = (counter 1 2 2 (queue '() '((mia . 2)) 0 1))
- apoi trec 5 minute, după care situația este:

- mia a ieșit de la C1, care a rămas goală  $\Rightarrow C1 = (\text{counter } 1 \ 0 \ 0 \ (\text{queue } '() \ '() \ 0 \ 0))$
- la C2 au trecut 5 minute  $\Rightarrow C2 = (\text{counter } 2 \ 9 \ 9 \ (\text{queue } '() \ '(\text{ana} \ . \ 14)) \ 0 \ 1))$
- C3 a rămas cum era: goală și neîntârziată  $\Rightarrow C3 = (\text{counter } 3 \ 0 \ 0 \ (\text{queue } '() \ '() \ 0 \ 0))$
- au părăsit magazinul, în ordine:  $'((1 \ . \ \text{mia}))$  (1 reprezintă indexul casei de la care a ieșit mia)
- ion se așază la C3  $\Rightarrow C3 = (\text{counter } 3 \ 7 \ 7 \ (\text{queue } '() \ '(\text{ion} \ . \ 7)) \ 0 \ 1))$
- C1 este întârziată cu 2 minute  $\Rightarrow C1 = (\text{counter } 1 \ 2 \ 2 \ (\text{queue } '() \ '() \ 0 \ 0))$
- apoi trec 7 minute, după care situația este:
  - întârzierea de la C1 s-a consumat  $\Rightarrow C1 = (\text{counter } 1 \ 0 \ 0 \ (\text{queue } '() \ '() \ 0 \ 0))$
  - la C2 au trecut 7 minute  $\Rightarrow C2 = (\text{counter } 2 \ 2 \ 2 \ (\text{queue } '() \ '(\text{ana} \ . \ 14)) \ 0 \ 1))$
  - ion a ieșit de la C3, care a rămas goală  $\Rightarrow C3 = (\text{counter } 3 \ 0 \ 0 \ (\text{queue } '() \ '() \ 0 \ 0))$
  - au părăsit magazinul, în ordine:  $'((1 \ . \ \text{mia}) \ (3 \ . \ \text{ion}))$

## Etapa 4

Etapa 4 oferă un exemplu interesant de utilizare a fluxurilor. Veți reimplementa TDA-ul queue pentru a obține un plus de performanță și, cu condiția să fi respectat bariera de abstractizare în etapa 3, funcțiile implementate anterior vor funcționa fără modificări pe noua reprezentare.

Din nou, rezolvarea etapei începe cu implementarea TDA-ului queue în fișierul **queue.rkt**.

Din motive de performanță detaliate în schelet, reținem câmpul `left` al structurii queue ca flux (în contrast cu reprezentarea ca listă din etapa 3). Definiția structurii nu se modifică:

```
(define-struct queue (left right size-l size-r))
```

- o adăugare în coadă este o adăugare în stiva `right` (ca înainte)
- o extragere din coadă este o extragere din stiva `left` (ca înainte)
- după fiecare operație enqueue sau dequeue trebuie menținut invariantul `size(left) ≥ size(right)`; astfel, niciun dequeue nu va găsi stiva `left` vidă
- când o operație enqueue sau dequeue produce situația `size(left) = size(right) - 1`, aplicăm o **rotație**:
  - mutăm "în mod leneș" toate elementele din `right` în `left`
  - ce înseamnă "leneș": elementele vor fi mutate, de fapt, unul câte unul, pe măsură ce extragem elemente din `left`, nu toate deodată (dacă s-ar muta deodată nu am rezolva problema complexității, ci doar am deplasa-o asupra altor operații)

Veți redefini interfața din etapa 3. Noile implementări depind de implementarea funcției de rotație:

```
(rotate left right Acc)
```

- `rotate` calculează (cu evaluare întârziată) rezultatul `left ++ (reverse right)`
- rotația se efectuează doar atunci când `size(left) = size(right) - 1`, așadar găsește un număr echilibrat de elemente în cele două stive
- la fiecare extragere din `left`, extragem ("pop") și un element din `right` pe care îl adăugăm ("push") în acumulatorul `Acc`
- când `left` devine goală, `right` conține un singur element (`size(left) = size(right) - 1`), iar `Acc` conține toate elementele aflate inițial în `right`, în ordine inversă; acum adăugăm elementul

din **right** la începutul **Acc** (în timp  $O(1)$ ), și acesta este exact conținutul cu care trebuie să reinițializăm stiva **left**

Exemplu:

```
(rotate (stream-cons 1 (stream-cons 2 (stream-cons 3 empty-stream)))
        '(7 6 5 4)
        empty-stream)
```

⇒ #<stream>

Mai precis, rezultatul este de forma:

```
(stream-cons 1
  (rotate (stream-cons 2 (stream-cons 3 empty-stream))
    '(6 5 4)
    (stream-cons 7 empty-stream)))
```

și, conform comportamentului constructorului **stream-cons**, apelul recursiv al funcției **rotate** este întârziat. Când accesăm restul acestui flux (de exemplu, la **dequeue**), evaluăm apelul întârziat, obținând un rezultat de forma **(stream-cons 2 (rotate ....))**, etc.

După ce ați finalizat implementarea TDA-ului, continuați implementarea în fișierul **etapa4.rkt**.

Față de etapa anterioară, simulatorul tratează două cereri noi:

- (**close index**) solicită închiderea casei cu indexul **index**, și redistribuirea clienților din coadă (cu excepția primului)
- (**open index**) solicită deschiderea casei cu indexul **index**

Apare distincția între case deschise și case închise: în această etapă, cererile de tip "așezare la o casă", respectiv "ensure" iau în considerare doar casele deschise. Modul în care reprezentați starea caselor (deschisă/închisă) este la alegerea voastră.

Exemplu pentru **ITEMS = 5**:

```
(serve '((ana 7) (mia 2) 5 (ion 8) (dan 6) (close 2) (delay 1 15) (ema 2) (open 2) 2 (geo 5) (close 1) (ensure 7))
        (list (empty-counter 1))
        (list (empty-counter 2) (empty-counter 3)))
```

- avem o casă fast (o numim **C1**) și două case slow (le numim **C2** și **C3**)
- când ilustrăm starea caselor:
  - o casă este o colecție de **index**, **tt**, **et** și **queue** (dar puteți modifica structura, dacă doriți)
  - vizualizăm elementele fluxurilor între acolade (în loc să scriem #<stream>, ceea ce nu este tocmai informativ)
- primele două cereri distribuie cei doi clienți astfel:
  - ana la **C2** ⇒ **C2 = (counter 2 7 7 (queue {(ana . 7)} '() 1 0))**
  - mia la **C1** ⇒ **C1 = (counter 1 2 2 (queue {(mia . 2)} '() 1 0))**
  - obs: în etapa trecută ana și mia apăreau ca elemente în stiva **right**; acum ele sunt în stiva **left**, deoarece s-a efectuat o rotație, necesară pentru menținerea invariantului  $size(left) \geq size(right)$
- apoi trec 5 minute, după care situația este:
  - mia a ieșit de la **C1**, care a rămas goală ⇒ **C1 = (counter 1 0 0 (queue {} '() 0 0))**
  - la **C2** au trecut 5 minute ⇒ **C2 = (counter 2 2 2 (queue {(ana . 7)} '() 1 0))**
  - **C3** a rămas cum era: goală și neîntârziată ⇒ **C3 = (counter 3 0 0 (queue {} '() 0 0))**

- următoarele două cereri distribuie cei doi clienți astfel:
  - ion la C3  $\Rightarrow C3 = (\text{counter } 3 \ 8 \ 8 \ (\text{queue } \{(\text{ion} \ . \ 8)\} \ '() \ 1 \ 0))$
  - dan la C2  $\Rightarrow C2 = (\text{counter } 2 \ 8 \ 2 \ (\text{queue } \{(\text{ana} \ . \ 7)\} \ '((\text{dan} \ . \ 6)) \ 1 \ 1))$
- C2 se închide  $\Rightarrow$  ana rămâne la C2, iar dan se mută la C3
  - $\Rightarrow C2 = (\text{counter } 2 \ 2 \ 2 \ (\text{queue } \{(\text{ana} \ . \ 7)\} \ '() \ 1 \ 0))$  și nu mai primește clienți
  - $\Rightarrow C3 = (\text{counter } 3 \ 14 \ 8 \ (\text{queue } \{(\text{ion} \ . \ 8)\} \ '((\text{dan} \ . \ 6)) \ 1 \ 1))$
- C1 este întârziată cu 15 minute  $\Rightarrow C1 = (\text{counter } 1 \ 15 \ 15 \ (\text{queue } \{\} \ '() \ 0 \ 0))$
- ema se așază la C3  $\Rightarrow C3 = (\text{counter } 3 \ 16 \ 8 \ (\text{queue } \{(\text{ion} \ . \ 8) \ <\text{flux-reevaluat-care-va-produce-dan-și-ema}>\} \ '() \ 3 \ 0))$ 
  - C2 are tt mai mic, însă C2 este închisă, deci se alege între C1 și C3
- C2 se deschide, fără să producă alte modificări
- apoi trec 2 minute, după care situația este:
  - întârzierea de la C1 s-a consumat parțial  $\Rightarrow C1 = (\text{counter } 1 \ 13 \ 13 \ (\text{queue } \{\} \ '() \ 0 \ 0))$
  - ana a ieșit de la C2  $\Rightarrow C2 = (\text{counter } 2 \ 0 \ 0 \ (\text{queue } \{\} \ '() \ 0 \ 0))$
  - la C3 au trecut 2 minute  $\Rightarrow C3 = (\text{counter } 3 \ 14 \ 6 \ (\text{queue } \{(\text{ion} \ . \ 8) \ <\text{flux...}>\} \ '() \ 3 \ 0))$
- geo se așază la C2  $\Rightarrow C2 = (\text{counter } 2 \ 5 \ 5 \ (\text{queue } \{(\text{geo} \ . \ 5)\} \ '() \ 1 \ 0))$
- C1 se închide, fără să producă alte modificări
- ensure compară media timpilor totali ai caselor deschise cu 7
  - $tt2 + tt3 = 5 + 14 = 19 \Rightarrow tt\text{-mediu} = 19 / 2 > 7$
  - tt1 nu participă la medie întrucât C1 este închisă
  - se adaugă o casă slow goală (C4)  $\Rightarrow tt\text{-mediu} = 19 / 3 \leq 7$  (deci ne oprim aici cu adăugarea)

Rezultat final:

```
(list
 '((1 . mia) (2 . ana))
 (cons 2 (queue #<stream> '() 1 0))
 (cons 3 (queue #<stream> '() 3 0)))
```

## Precizări

- Scheletul fiecărei etape va conține unul sau mai multe fișiere .rkt în care trebuie să lucrați, plus fișierul **checker.rkt** pe care îl veți folosi doar pentru testare (rulând codul fără să îl modificați).
- Fiecare etapă (o arhivă .zip cu fișierele în care ați lucrat, plus eventualele fișiere care sunt solicitate de acestea cu "require") se va încărca pe vmchecker. Testele de vmchecker sunt aceleași cu cele din checker.rkt.
- În fiecare etapă veți avea de implementat o serie de funcții, în sprijinul cărora vă puteți defini oricând **funcții ajutătoare** (dacă nu se interzice asta în mod explicit). Atunci când există restricții asupra implementării funcției din cerință, aceleași restricții trebuie respectate și de eventualele funcții ajutătoare definite de voi.
- Dacă doriți să rezolvați exerciții din etapa curentă care depind de exerciții din etapele anterioare pe care nu le-ați rezolvat, puteți semnala acest lucru responsabilului de temă, care vă va pune la dispoziție o rezolvare pentru acele exerciții astfel încât să puteți continua tema. Odată ce alegeți această variantă, renunțați la dreptul de a mai trimite cu întârziere etapa pentru care ați solicitat parțial sau integral rezolvarea. Puteți solicita rezolvări doar pentru exercițiile din etapele anterioare, nu și pentru cele din etapa curentă. Dacă implementați un exercițiu din etapa curentă pe baza unui

alt exercițiu din etapa curentă pe care nu l-ați rezolvat, se va lua în calcul punctajul dat de checker, chiar dacă implementarea ar funcționa în caz că exercițiul nerezolvat ar funcționa.

- Tema este o temă de programare funcțională - pentru care folosim Racket. Racket este un limbaj multiparadigmă, care conține și elemente "ne-funcționale" (de exemplu proceduri cu efecte laterale), pe care **nu** este permis să le folosiți în rezolvare.
- Pentru fiecare etapă, checker-ul vă oferă un punctaj între 0 și 120 de puncte. Nota 100 corespunde punctajului maxim pe etapă, iar orice surplus se transformă într-un bonus proporțional.
- Veți prezenta tema asistentului, care poate modifica punctajul dat de checker dacă observă nereguli precum răspunsuri hardcodate, proceduri cu efecte laterale, implementări neconforme cu restricțiile din enunț.
- Temele implementează o politică de **zero toleranță pentru copiere**. Cât timp respectați politica de colaborare și utilizare a internetului descrisă în regulamentul materiei, nu aveți motive de îngrijorare. Contăm pe voi să nu recurgeți la mijloace nepermise, și vă stăm la dispoziție pentru orice neclaritate.

## Resurse

---

- etapa 4
- etapa 3
- etapa 2
- etapa 1
- tutorial structuri

## Changelog

---

- 27.03 (ora 23:12) - Am publicat etapa 4.
- 19.03 (ora 10:32) - Am publicat etapa 3.
- 13.03 (ora 08:10) - Am publicat etapa 2.
- 09.03 (ora 15:30) - Am publicat etapa 1.

## Referințe

---

- Simple and Efficient Purely Functional Queues and Deques  
[<http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.47.8825&rep=rep1&type=pdf>]